

Name: Oratile

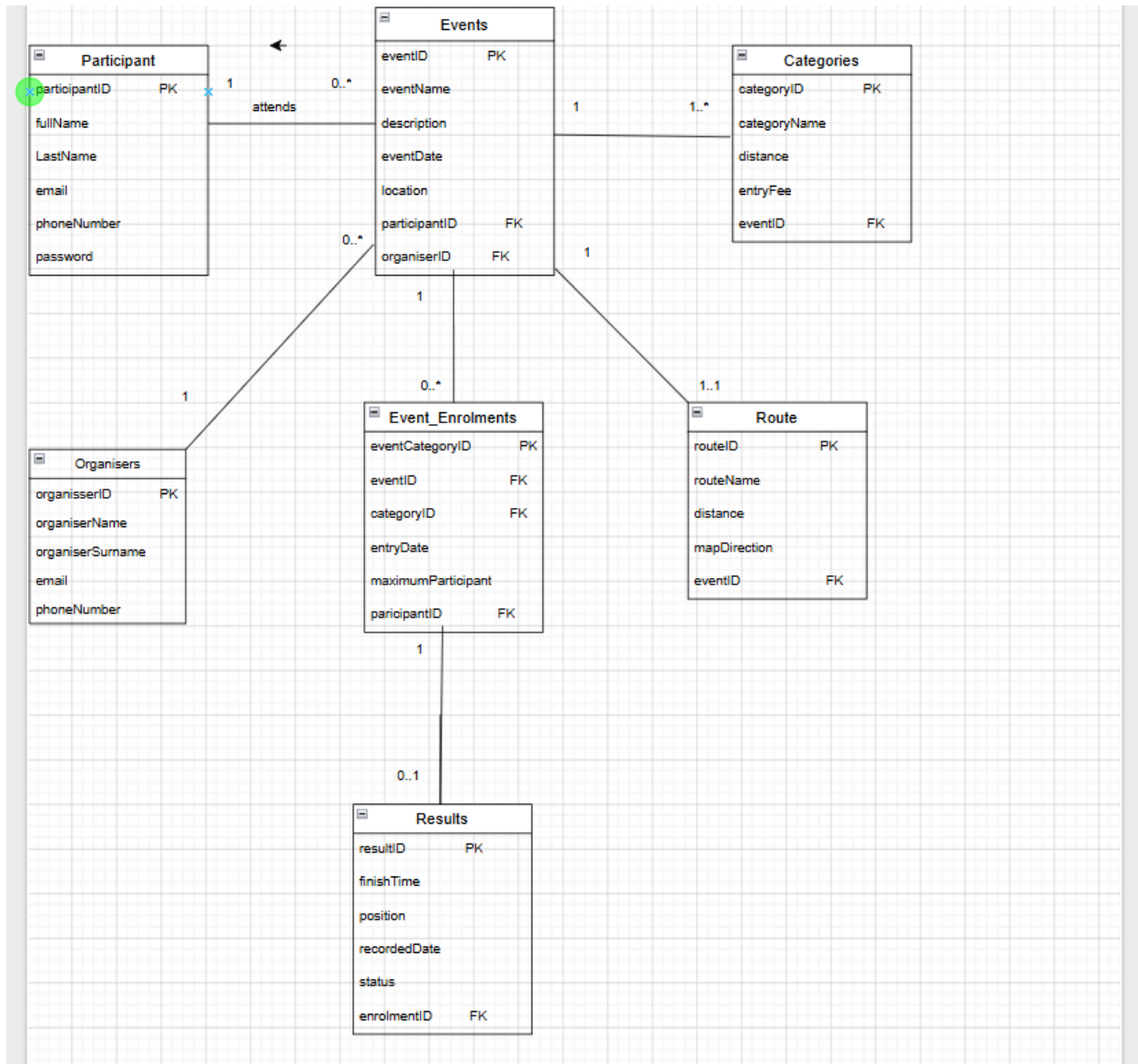
Surname: Matlhabane

Student Number: ST10486463

Module Name: Programming 2B

Module Code: Prog6212

ERD



SECTION B: API table

1.Authentication

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
Post	/api/auth/register	Registers a new user as a Participant or Organizer	None (Public)	Full name, last name, email, password, phone number,role	201 Created – user registered successfully
Post	/api/auth/login	Allows users to log in according to their assigned role.	None (Public)	Email, password	200 OK – Login successful and authentication token returned.

2. User profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/profile	Allows a user to view their own profile information.	Participant/organiser	None	200 OK – User profile returned
PUT	/api/users/profile	Update the authenticated user's profile information	Any logged in users/organizer	Updated profile information	200 OK – Profile updated successfully

3.Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Allows both roles to view available events.	Participant / Organisers	None	200 OK – List of events returned

GET	/api/events{id}	Allows both roles to view the details of a specific event.	Participant / Organiser	None	200 OK- Event details returned
POST	/api/evevents	Allows an organiser to create a new event.	Organiser	Event name, description, date, location, distance, and event type.	201 Created – Event created successfully
PUT	/api/events{Id}	Allows organiser to update an existing even	Oragniser	Updated event details	200 OK – Event Updated successfully
Delete	/api/event/{id}	Allows an organiser to delete an event.	Oragniser	None	204 No Content – Event deleted successfully

4.Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Allows users to view categories available for an event.	Participant/ Organiser	None	200 OK – Available categories returned
POST	/api/events/{eventId}/categories	Allows an organiser to define an age or distance category	Oragniser	Category name, age/distance and maximum	201 Created – Category created successfully

		for an event.		participants.	
PUT	/api/categories/{id}	Allows an organiser to update a category.	Organiser	Updated category information.	200 OK – Category updated successfully
DELETE	/api/categories/{id}	Allows an organiser to delete a category.	Organiser	None	204 No content – Category deleted successfully

5.Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments	Allows a participant to enter an event by selecting a category.	Participant	Selected category and payment status.	201 Created – Event enrolment created successfully.
GET	/api/enrolments/my	Allows an organiser to view all participants enrolled in their event.	Organiser	None	200 OK – List of event enrolments returned.
GET	/api/events/{eventId}/enrolments	Allows a participant to view their own event enrolment.	Participant	None	200 OK – Enrolment details returned.

DELETE	/api/events/{eventId}/enrolments	Allows a participant to cancel their event enrolment.	Participant	None	204 No Content – Enrolment cancelled successfully.
--------	----------------------------------	---	-------------	------	--

6.Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/results	Allows an organiser to record a participant's finishing time and position.	Organiser	Finish time, finishing position, recorded date and status.	201 Created – Result recorded successfully.
GET	/api/results/my	Allows a participant to view their own race results.	Participant	None	200 OK – Participant's results returned.
GET	/api/events/{eventId}/results	Allows an organiser to view results for participants in their event.	Organiser	None	200 OK – Event results returned.
PUT	/api/results/{resultId}	Allows an organiser to update a participant's result.	Organiser	Updated finish time, position and status.	200 OK – Result updated successfully.

7. Routes

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/route	Allows users to view route information for an event.	Participant/ Organiser	None	200 OK – Route name, distance and map directions returned.
POST	/api/events/{eventId}/route	Allows an organiser to add route information to an event.	Organiser	Route name, distance and map directions.	201 Created – Route created successfully.
PUT	/api/routes/{routeId}	Allows an organiser to update route information.	Organiser	Updated route information.	200 OK – Route updated successfully.

Section C: SQL

```

1  create database RaceDay;
2  use RaceDay;
3
4
5  CREATE TABLE Organisers (
6      organiserID    INT IDENTITY(10,1) PRIMARY KEY,
7      organiserName  VARCHAR(250)    NOT NULL,
8      organiserSurname VARCHAR(250)    NOT NULL,
9      email          VARCHAR(250)    NOT NULL UNIQUE,
10     phoneNumber    VARCHAR(250)    NULL
11 );
12
13 CREATE TABLE Participant (
14     participantID  INT IDENTITY(20,1) PRIMARY KEY,
15     fullName      VARCHAR(250)    NOT NULL,
16     LastName      VARCHAR(250)    NOT NULL,
17     email         VARCHAR(250)    NOT NULL UNIQUE,
18     phoneNumber   VARCHAR(250)    NULL,
19     password      VARCHAR(250)    NOT NULL
20 );
21
22 CREATE TABLE Events (
23     eventID        INT IDENTITY(30,1) PRIMARY KEY,
24     eventName      VARCHAR(200)    NOT NULL,
25     description    VARCHAR(255)    NULL,
26     eventDate      DATE             NOT NULL,
27     location       VARCHAR(250)    NOT NULL,
28     participantID  INT             NULL,
29     organiserID    INT             NOT NULL,
30
31     FOREIGN KEY (participantID) REFERENCES Participant(participantID),
32     CONSTRAINT FK_Events_Organisers
33     FOREIGN KEY (organiserID) REFERENCES Organisers(organiserID)
34 );
35
36 CREATE TABLE Categories (
37     categoryID     INT IDENTITY(1,1) PRIMARY KEY,
38     categoryName   VARCHAR(50)    NOT NULL,
39     distance       DECIMAL(6,2)    NOT NULL,
40     entryFee       DECIMAL(8,2)    NOT NULL DEFAULT 0,
41     eventID        INT             NOT NULL,
42
43     FOREIGN KEY (eventID) REFERENCES Events(eventID)
44 );
45
46 CREATE TABLE Route (
47     routeID        INT IDENTITY(60,1) PRIMARY KEY,
48     routeName      VARCHAR(100)    NOT NULL,
49     distance       DECIMAL(6,2)    NOT NULL,
50     mapDirection   VARCHAR(255)    NULL,
51     eventID        INT             NOT NULL UNIQUE,
52
53     FOREIGN KEY (eventID) REFERENCES Events(eventID)
54 );

```

```

CREATE TABLE Event_Enrolments (
    eventCategoryID INT IDENTITY(100,1) PRIMARY KEY,
    eventID INT NOT NULL,
    categoryID INT NOT NULL,
    entryDate DATE NOT NULL DEFAULT GETDATE(),
    maximumParticipant INT NOT NULL,
    participantID INT NOT NULL,

    FOREIGN KEY (eventID) REFERENCES Events(eventID),

    FOREIGN KEY (categoryID) REFERENCES Categories(categoryID),

    FOREIGN KEY (participantID) REFERENCES Participant(participantID),

    UNIQUE (participantID, categoryID)
);

CREATE TABLE Results (
    resultID INT IDENTITY(1,1) PRIMARY KEY,
    finishTime TIME NULL,
    position INT NULL,
    recordedDate DATE NULL,
    status VARCHAR(20) NOT NULL
    CONSTRAINT CHK_Results_Status
    CHECK (status IN ('Finished', 'DNF', 'DNS', 'DSQ')), enrolmentID INT NOT NULL UNIQUE,
    CONSTRAINT FK_Results_Enrolments
    FOREIGN KEY (enrolmentID) REFERENCES Event_Enrolments(eventCategoryID)
);

```

```

-- Organisers (2)
INSERT INTO Organisers (organiserName, organiserSurname, email, phoneNumber) VALUES
('Thabo', 'Mokoena', 'thabo.mokoena@raceday.co.za', '0821234567'),
('Sarah', 'van der Merwe', 'sarah.vdm@raceday.co.za', '0827654321');

-- Participants (2)
INSERT INTO Participant (fullName, LastName, email, phoneNumber, password) VALUES
('Lindiwe', 'Nkosi', 'lindiwe.nkosi@gmail.com', '0839876543', 'HASHED_PW_1'),
('James', 'Botha', 'james.botha@gmail.com', '0845551234', 'HASHED_PW_2');

-- Events (3) - organiserID 1 and 2 from above
INSERT INTO Events (eventName, description, eventDate, location, participantID, organiserID) VALUES
('Benoni Park Run Challenge', 'Annual community road running event', '2026-10-10', 'Benoni, Gauteng', 1, 1),
('Pretoria Cycle Classic', 'Charity cycling event through the city', '2026-11-15', 'Pretoria, Gauteng', 2, 2),
('Soweto Heritage Walk', 'Community fun walk celebrating local heritage', '2026-12-05', 'Soweto, Gauteng', NULL, 1);

-- Routes (one per event, 1..1)
INSERT INTO Route (routeName, distance, mapDirection, eventID) VALUES
('Benoni Main Loop', 10.00, 'https://maps.raceday.co.za/benoni-loop', 1),
('Pretoria City Circuit', 42.00, 'https://maps.raceday.co.za/pretoria-circuit', 2),
('Soweto Heritage Trail', 5.00, 'https://maps.raceday.co.za/soweto-trail', 3);

```

```

INSERT INTO Categories (categoryName, distance, entryFee, eventID) VALUES
('10km Run', 10.00, 150.00, 1),
('5km Fun Walk', 5.00, 80.00, 1),
('42km Cycle', 42.00, 300.00, 2),
('5km Fun Walk', 5.00, 80.00, 3),
('21km Half Marathon', 21.10, 200.00, 3);
INSERT INTO Event_Enrolments (eventID, categoryID, entryDate, maximumParticipant, participantID) VALUES
(1, 1, '2026-09-05', 500, 1),
(1, 2, '2026-09-06', 300, 2),
(2, 3, '2026-09-10', 250, 1),
(3, 5, '2026-09-12', 200, 2);

INSERT INTO Results (finishTime, position, recordedDate, status, enrolmentID) VALUES
('00:52:31', 1, '2026-10-10', 'Finished', 1);

```

	participantID	fullName	LastName	email	phoneNumber	password
1	20	Lindiwe	Nkosi	lindiwe.nkosi@gmail.com	0839876543	HASHED_PW_1
2	21	James	Botha	james.botha@gmail.com	0845551234	HASHED_PW_2

	organiserID	organiserName	organiserSurname	email	phoneNumber
1	10	Thabo	Mokoena	thabo.mokoena@raceday.co.za	0821234567
2	11	Sarah	van der Merwe	sarah.vdm@raceday.co.za	0827654321

Links

[ST10486463/RaceDay](#)

[RaceDay-Prog2B - YouTube](#)

References

Gosselin, Don, et al. *PHP Programming With MySQL*. don gosselin, 2011.

W3Schools (2019). *SQL Tutorial*. [online]
W3schools.com.